

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Applied Sciences · Department of Computer Science

Solution Book 5

SCS 1101, December 2004 (Engineering students)

SCS 1101 Introduction to Computer Science and Programming

Past-paper solutions with marks allocated, algorithms, textbook flowcharts, exam-ready code and a civil & water engineering lens

Engineering students · 2 from Section A + 3 from Section B · 20 marks each

Book 5 of 8 · Prepared October 2026

Index — tap a tab or a line to jump

Every chapter starts with the coloured tab bar below; tap any tab to jump straight to that question. The page footer has a link back to this index. In Word, the Navigation Pane shows the same chapters.

Guide How to use this book

Glance At a glance

Q1 Question 1 [20]

- ▸ 1(a) Explain, with examples [2 each = 10]
- ▸ 1(b) Advantages of networking computers [5]
- ▸ 1(c) Benefits and drawbacks of personal computers [5]

Q2 Question 2 [20]

- ▸ 2(a)(i) Convert 13.5625 to binary [1]
- ▸ 2(a)(ii) CAD × B0D in hexadecimal [1]
- ▸ 2(b) Describe the computer generations [8]
- ▸ 2(c) Components needed to assemble a computer from scratch [10]

Q3 Question 3 [20]

- ▸ 3(a) How are indexed sequential files accessed? [2]
- ▸ 3(b) Laser vs dot-matrix printers [4]
- ▸ 3(c) Computer virus and symptoms of infection [4]
- ▸ 3(d) Five types of high-level programming languages, with examples [10]

Q4 Question 4 [20]

- ▸ 4(a) Evaluate, with a = 8, b = 3, c = −5 (all int) [5]
- ▸ 4(b) Program: read a string, print it reversed and its length [5]
- ▸ 4(c) Program: sum, mean, deviations and standard deviation of m values [10]

Q5 Question 5 [20]

- ▸ Menu-driven perimeter program with functions, plus a flowchart

Q6 Question 6 [20]

- ▸ 6(a) Semantic error, with an example [2]
- ▸ 6(b) Syntax of the switch statement [3]
- ▸ 6(c) Valid or invalid? [5]
- ▸ 6(d) Program: each value from the largest down, with count and positions [10]

Q7 Question 7 [20]

- ▸ 7(a) What is an infinite loop? Example [2]
- ▸ 7(b) Output of the nested loop [3]
- ▸ 7(c) Program: rectangle area using pointers [5]
- ▸ 7(d) Program: help a child count with asterisks [10]

Refs Sources for this book

Links Source register

How to use this book

This book is one of eight stand-alone solution books for NUST SCS 1101 past papers. It solves **SCS 1101, December 2004 (Engineering students)** question by question, with the marks the examiner allocates shown in square brackets, e.g. [2].

The six-block answer template

Every answer in this book (and in all eight books of the set) is built from the same six blocks, in the same order. Theory answers use blocks 1, 2, 5 and 6; programming answers use all six.

1. **Like you're 3** — one or two sentences with a toy, a bath or a water tap.
2. **Model answer** — the exam wording, with the mark for each point in brackets, e.g. [2]. Write it like this and you collect every mark.
3. **Algorithm + code** — numbered pseudocode first, then C code that follows the **H-D-I-P-O-R** recipe.
4. **Flowchart or structure chart** — textbook (ANSI) symbols. Programs built from functions also get a structure (hierarchy) chart.
5. **Engineering lens** — the same idea in a civil and water engineering setting, using the profession's vocabulary.
6. **Examiner's tip** — where students usually lose marks.

The H-D-I-P-O-R code recipe — "**H**appy **D**ucks **I**n **P**onds **O**nly **R**elax". Every exam program is these six steps, in this order:

```c
/* H - Header: what the program does, and the libraries */
#include <stdio.h>

int main(void)
{
    /* D - Declare: every variable, with a sensible type and a starting value */
    /* I - Input:   printf a prompt, then scanf with & */
    /* P - Process: the if-ladder, loop or formula */
    /* O - Output:  printf the result with units and decimals */
    /* R - Return:  */ return 0;
}
```

The recipe is worth marks by itself: examiners give separate marks for headers, declarations, input, processing and output. A half-finished program laid out this way still scores; a clever one-liner without structure often does not.

Algorithm style used in every book:

```
Step 1: Start
Step 2: Read <inputs>
Step 3: <process, one action per step>
Step 4: Display <outputs>
Step 5: Stop
```

Flowchart symbols (ANSI):

Symbol	Shape	Use	Colour in this book
Terminator	Rounded box (stadium)	Start / Stop	Green

Symbol	Shape	Use	Colour in this book
Input/Output	Parallelogram	Read or display	Blue
Process	Rectangle	Calculation or assignment	Amber
Decision	Diamond	Yes/no test, two exits	Red
Predefined process	Rectangle with double sides	Call to a function	Purple
Flow line	Arrow	Order of steps	Dark grey arrow

The smallest possible program, drawn with these symbols:

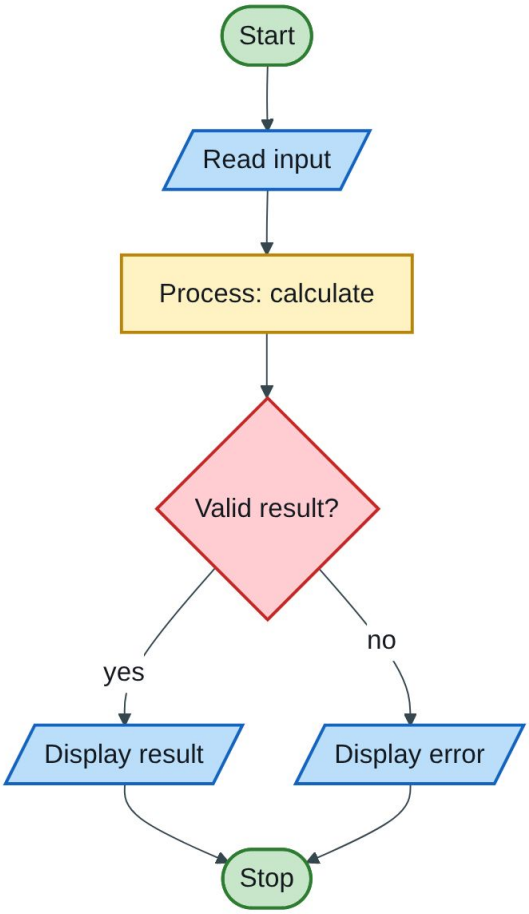


Figure 5.1 — Flowchart

The engineering lens. Programming questions get a civil and water engineering version where it fits, for example:

- tariff ladders become **increasing block tariffs** for water;
- array statistics become **rainfall** or **concrete cube strength** analysis;
- geometry programs become **cross-section, wetted perimeter and hydraulic radius** calculations.

Write the exam version in the exam; the engineering version is for understanding and for your later modules.

How the solutions were researched

Every fact a solution relies on is traced to a source in one of four tiers; each source has a short key (e.g. [MIT-2]) so answers can cite it without long links.

1. **Tier 1 — the course itself.** The SCS 1101 outline and the past papers decide *what* to answer and how marks are split.

2. **Tier 2 — university teaching material.** MIT OpenCourseWare and UCLA course pages for programming and algorithms; university histories for milestones.
3. **Tier 3 — standards and research.** NIST for cloud definitions; peer-reviewed and arXiv papers, including Wits research on cloud adoption in South Africa.
4. **Tier 4 — standard textbook knowledge.** Definitions every textbook agrees on (OS functions, topologies, printers) are written from the textbooks your outline lists, without a web citation.

A claim is only cited when a source was actually opened; anything from tier 4 stays general and uncited.

At a glance

3 hours, 20 marks per question. Answer **5 questions: any 2 from Section A (Q1–Q3) and any 3 from Section B (Q4–Q7)**. This was the engineering-students paper, so Section B is pure C programming: practise all four of its questions.

Question	What it tests	Section	Pick?
Q1	Hardware, software, firmware, ROM, RAM; networking; PCs	A	Yes
Q2	13.5625 to binary; CAD × B0D; generations; assembling a PC	A	Yes
Q3	Indexed sequential files; printers; viruses; five language types	A	Backup
Q4	Expressions; reverse a string; mean and standard deviation	B	Yes
Q5	Menu-driven perimeter program with functions and a flowchart	B	Yes, one long program
Q6	Semantic error; switch syntax; valid or invalid; max-occurrence program	B	Yes
Q7	Infinite loop; nested-loop output; pointers; counting program	B	Yes

Question 1 [20]

1(a) Explain, with examples [2 each = 10]

Like you're 3: hardware is the toy you can touch; software is the game rules inside it; firmware is rules glued permanently into the toy; ROM is the book that cannot be written in; RAM is the whiteboard wiped every night.

Term	Explanation [1]	Example [1]
Hardware	The physical, touchable parts of a computer system.	Keyboard, CPU, hard disk, monitor
Software	Programs and data that instruct the hardware; it cannot be touched.	Windows, Microsoft Excel, AutoCAD
Firmware	Software permanently stored in non-volatile memory (ROM/flash) inside a device, controlling its basic functions.	PC BIOS/UEFI; the program inside a printer or a PLC
ROM	Read-only, non-volatile memory holding permanent start-up instructions.	BIOS chip on the motherboard
RAM	Read/write, volatile main memory holding the programs and data in use.	8 GB DDR4 memory module

1(b) Advantages of networking computers [5]

Any five, 1 each:

1. **Sharing hardware** — one printer, scanner or plotter serves the whole office.
2. **Sharing files and data** — a central server holds one up-to-date copy of drawings and records.
3. **Shared internet connection** — one link serves everyone.
4. **Communication** — email, messaging and video calls between users.
5. **Central backup and security** — data is backed up and protected in one place.
6. **Lower software cost** — network or site licences instead of one per PC.

1(c) Benefits and drawbacks of personal computers [5]

Benefits	Drawbacks
Affordable computing power for one person	Health problems: eye strain, back pain, repetitive strain injury
Fast, accurate calculation and word processing	Security risks: viruses, theft of data
Easy storage and retrieval of large amounts of data	Rapid obsolescence: needs frequent upgrades
Access to the internet and email	Dependence: work stops when the PC fails or power is cut
Supports productivity software (CAD, spreadsheets)	E-waste and energy use

About 1 mark per explained point; give at least three benefits and two drawbacks.

Engineering lens: firmware runs every modern instrument on site, from total stations and GPS receivers to flow meters and pump drives, so “firmware update” is routine maintenance vocabulary.

Question 2 [20]

2(a)(i) Convert 13.5625 to binary [1]

Like you're 3: the whole part is shared out by halving (keep the leftovers); the fraction part is doubled again and again (keep what spills over 1).

Whole part: 13 ÷ 2	Quotient	Remainder
13 ÷ 2	6	1
6 ÷ 2	3	0
3 ÷ 2	1	1
1 ÷ 2	0	1

Read the remainders upwards: 13 = 1101.

Fraction part: × 2	Result	Bit taken
0.5625 × 2	1.125	1
0.125 × 2	0.25	0
0.25 × 2	0.5	0
0.5 × 2	1.0	1 (stop: fraction is 0)

Read downwards: 0.5625 = 0.1001. **$13.5625 = 1101.1001_2$**. Check: 8 + 4 + 1 + 0.5 + 0.0625 = 13.5625.

2(a)(ii) CAD × B0D in hexadecimal [1]

Like you're 3: long multiplication as in primary school, but you carry every 16 instead of every 10.

Row 1: CAD × D (D = 13): D × D = 169 = A9 → write 9, carry A; A × D + A = 140 = 8C → write C, carry 8; C × D + 8 = 164 = A4 → write A4. **Row 1 = A4C9**.

Row 2: CAD × 0 (the sixteens digit) = 0.

Row 3: CAD × B (B = 11), shifted two places: D × B = 143 = 8F → write F, carry 8; A × B + 8 = 118 = 76 → write 6, carry 7; C × B + 7 = 139 = 8B → write 8B. **Row 3 = 8B6F00**.

Add: 8B6F00 + A4C9 = **$8C13C9_{16}$**.

Check in decimal: CAD = 3245, B0D = 2829, and 3245 × 2829 = 9 180 105 = $8C13C9_{16}$.

2(b) Describe the computer generations [8]

Generation	Period	Key technology	Characteristics	Examples
1st	1940s–1956	Vacuum tubes	Huge, hot, unreliable; machine language; punched cards	ENIAC, UNIVAC I, MIT Whirlwind
2nd	1956–1963	Transistors	Smaller, faster, cooler; assembly, early FORTRAN and	IBM 1401

Generation	Period	Key technology	Characteristics	Examples
			COBOL; magnetic-core memory	
3rd	1964–1971	Integrated circuits	Smaller and cheaper; keyboards and monitors; operating systems; multiprogramming	IBM System/360
4th	1971–today	Microprocessors (VLSI)	Personal computers, GUIs, networks and the internet	IBM PC, Apple Macintosh
5th	Present and future	AI, parallel processing, ULSI	Natural-language processing, machine learning, voice assistants	Modern AI systems

About 1.5 marks per generation: technology plus one characteristic and an example.

2(c) Components needed to assemble a computer from scratch [10]

Like you're 3: building a computer is building a body: the brain (CPU), short-term memory (RAM), long-term memory (disk), the skeleton connecting everything (motherboard), the heart pumping power (power supply), and the skin (case).

Name [1] and function [1], five or more:

1. **Motherboard** — main circuit board that connects every component through its buses and slots.
2. **CPU (processor)** — executes instructions; the "brain".
3. **RAM** — holds running programs and data.
4. **Storage (SSD or hard disk)** — keeps the operating system and files permanently.
5. **Power supply unit (PSU)** — converts mains AC to the low-voltage DC the parts need.
6. **Case and cooling** — houses and protects the parts; fans or heat sinks remove heat.
7. **Graphics card** (or integrated graphics) — produces the display signal.
8. **Network card / Wi-Fi** — connects to a LAN or the internet.
9. **Input and output devices** — keyboard, mouse, monitor.
10. **Operating system** — software needed before the machine is usable.

Engineering lens: a workstation for BIM or finite-element analysis is specified like a structural design: a strong CPU, plenty of RAM and a good graphics card are the load-bearing elements; the rest just has to be adequate.

Question 3 [20]

3(a) How are indexed sequential files accessed? [2]

Like you're 3: like a dictionary with letter tabs on the side: jump to the right tab, then read page by page.

Records are stored **in key order** with an **index** of keys and their disk positions [1]. To find a record, the system looks up the key in the index, jumps straight to that block (direct access), then reads sequentially within the block; the whole file can also be read in order (sequential access) [1].

3(b) Laser vs dot-matrix printers [4]

Point	Laser printer	Dot-matrix printer
Type	Non-impact	Impact
How it prints	A laser draws the page on a charged drum; toner sticks and is fused by heat	Pins strike an inked ribbon, forming characters from dots
Quality and speed	High quality; fast, a page at a time	Low quality; slow, character by character
Noise and copies	Quiet; cannot do carbon copies	Noisy; can print multipart forms

Similarity: both are printers that produce hard copy from a computer. 1 mark per row of correct contrast.

3(c) Computer virus and symptoms of infection [4]

Definition [1]: a virus is a malicious program that attaches itself to files or programs, copies itself to other files and computers, and damages data or slows the system.

Symptoms [3, any three]: the computer runs unusually slowly; files disappear, are corrupted or change size; unexpected pop-ups or messages; programs crash or will not open; unusual disk or network activity; the antivirus is disabled; strange new files or shortcuts appear on USB drives.

3(d) Five types of high-level programming languages, with examples [10]

Like you're 3: languages are different kinds of instructions for the same robot: a step-by-step recipe, building with Lego objects, a maths formula, a list of facts and rules, or quick sticky notes.

Type [1]	What it is, with an example [1]
Procedural (imperative)	Program is a sequence of steps grouped into procedures/functions. C, Pascal, FORTRAN.
Object-oriented	Program is built from objects that combine data and methods; supports inheritance and polymorphism. C++, Java, Python.
Functional	Program is built from mathematical functions without changing variables. Haskell, Lisp.
Logic (declarative)	Program states facts and rules; the system works out the answer. Prolog.

Type [1]	What it is, with an example [1]
Scripting	Interpreted languages for automating tasks and the web. Python, JavaScript, PHP.

Alternative accepted classification by purpose: scientific (FORTRAN), business (COBOL), educational (BASIC, Pascal), systems (C), database query languages (SQL).

Question 4 [20]

4(a) Evaluate, with a = 8, b = 3, c = −5 (all int) [5]

Like you're 3: with whole numbers, C chops off any fraction and keeps only the whole part (towards zero), so 24 ÷ −5 = −4.8 becomes −4.

Expression	Working	Value [1 each]
(i) `a % b`	8 ÷ 3 = 2 remainder **2**	**2**
(ii) `a * b / c`	left to right: 8 × 3 = 24; 24 / −5 = −4.8 → truncated	**−4**
(iii) `a * (b / c)`	brackets first: 3 / −5 = −0.6 → 0; 8 × 0	**0**
(iv) `2 * b + 3 * (a - c)`	6 + 3 × (8 − (−5)) = 6 + 3 × 13 = 6 + 39	**45**
(v) `a / c`	8 / −5 = −1.6 → truncated	**−1**

Note how (ii) and (iii) differ only in brackets: integer division happens at a different moment.

4(b) Program: read a string, print it reversed and its length [5]

Like you're 3: write the word on beads, then read the beads from the last one back to the first, and count them.

```c
/* H: print a string backwards and give its length */
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[100];                                  /* D */
    int len, i;

    printf("Enter a string: ");                   /* I */
    fgets(s, sizeof s, stdin);
    len = strlen(s);
    if (len > 0 && s[len - 1] == '\n')           /* remove the Enter key */
        s[--len] = '\0';

    printf("Reversed: ");                         /* P + O */
    for (i = len - 1; i >= 0; i--)
        putchar(s[i]);
    printf("\nLength: %d\n", len);
    return 0;                                     /* R */
}
```

Test: `Civil Eng` → `gnE liviC`, length 9. Marks: reading the string [1], finding the length [1], reverse loop [2], output [1].

4(c) Program: sum, mean, deviations and standard deviation of m values [10]

Like you're 3: find the middle (mean) of all the numbers, see how far each one is from the middle (deviation), then work out the "typical distance" from the middle (standard deviation).

Formulas: $\bar{x} = (x_1 + \ldots + x_m) / m$; $d_i = x_i - \bar{x}$; $s = \sqrt{((d_1^2 + \ldots + d_m^2) / m)}$.

Algorithm:

```
Step 1: Start
Step 2: Read m, then x[1..m]; sum = sum + each x
Step 3: mean = sum / m
Step 4: For each i: d[i] = x[i] - mean; sumSq = sumSq + d[i]^2; display d[i]
Step 5: s = sqrt(sumSq / m)
Step 6: Display sum, mean, s
Step 7: Stop
```

Flowchart:

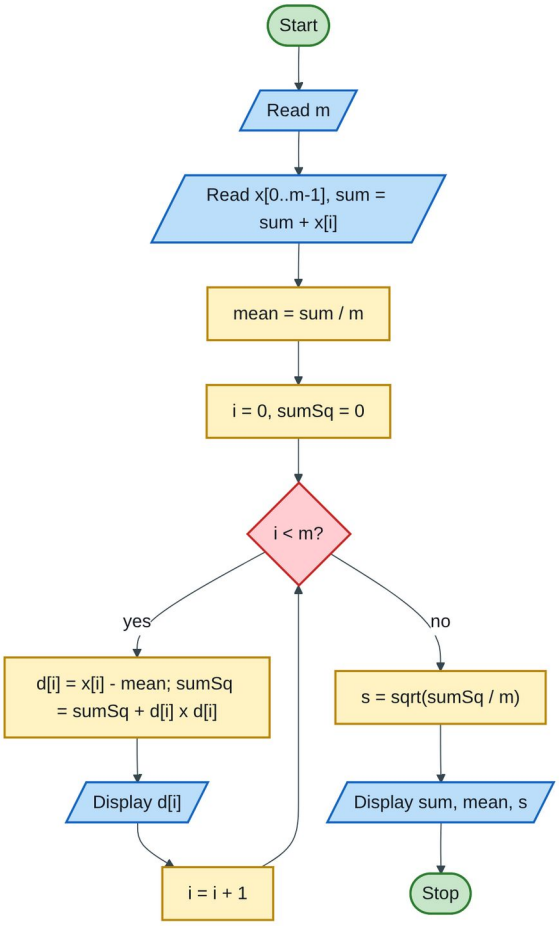


Figure 5.2 — Flowchart

Code (H-D-I-P-O-R):

```c
/* H: sum, mean, deviations and standard deviation of m float values */
#include <stdio.h>
#include <math.h>                       /* sqrt; compile with -lm */
#define MAX 100

int main(void)
{
    float x[MAX], d[MAX];                /* D */
    float sum = 0.0f, mean, sumSq = 0.0f, s;
    int m, i;

    printf("How many values (1-%d)? ", MAX);   /* I */
    scanf("%d", &m);
```

```
    if (m < 1 || m > MAX) {
        printf("Invalid m\n");
        return 1;
    }
    for (i = 0; i < m; i++) {
        printf("x[%d] = ", i + 1);
        scanf("%f", &x[i]);
        sum += x[i];
    }

    mean = sum / m;                          /* P */
    printf("\n  i        x    d = x - mean\n");
    for (i = 0; i < m; i++) {
        d[i] = x[i] - mean;
        sumSq += d[i] * d[i];
        printf("%3d %8.2f %12.2f\n", i + 1, x[i], d[i]);   /* O: deviations */
    }
    s = sqrt(sumSq / m);

    printf("\nSum                = %.2f\n", sum);
    printf("Mean               = %.2f\n", mean);
    printf("Standard deviation = %.3f\n", s);
    return 0;                                /* R */
}
```

Test: 2, 4, 4, 4, 5, 5, 7, 9 → sum 40, mean 5.00, deviations −3, −1, −1, −1, 0, 0, 2, 4; squares add to 32; s = √(32/8) = **2.000**.

Marking guide [10]: arrays and variables [1]; reading m and the values [2]; sum and mean [2]; deviations calculated and printed [2]; sum of squares and `sqrt` [2]; output [1].

Engineering lens: this is the statistics of **concrete quality control**. For a set of cube results, the standard deviation s measures how consistent the batching is; mixes are designed for a **target mean strength = characteristic strength + 1.64 s** (the "margin"), so a site with poor control (large s) must batch stronger, costlier concrete. The same program summarises **annual rainfall** variability for hydrology.

Examiner's tip: include `<math.h>` for `sqrt`, and remember that GCC on Linux needs `-lm` when linking.

Question 5 [20]

Menu-driven perimeter program with functions, plus a flowchart

Like you're 3: a friendly robot shows a picture menu of shapes. You point at one, it asks for the sizes it needs, tells you the distance all the way round, then asks "Another one?" until you say no.

Formulas: square P = 4a; rectangle P = 2(l + w); circle P = 2πr; right-angled triangle P = a + b + √(a² + b²) (Pythagoras gives the hypotenuse).

Structure chart (requirement (iv): functions called from `main`):

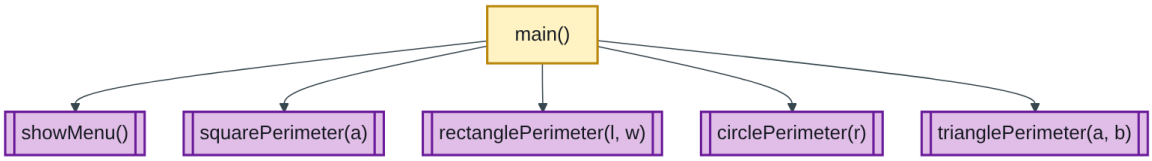


Figure 5.3 — Structure chart

Flowchart:

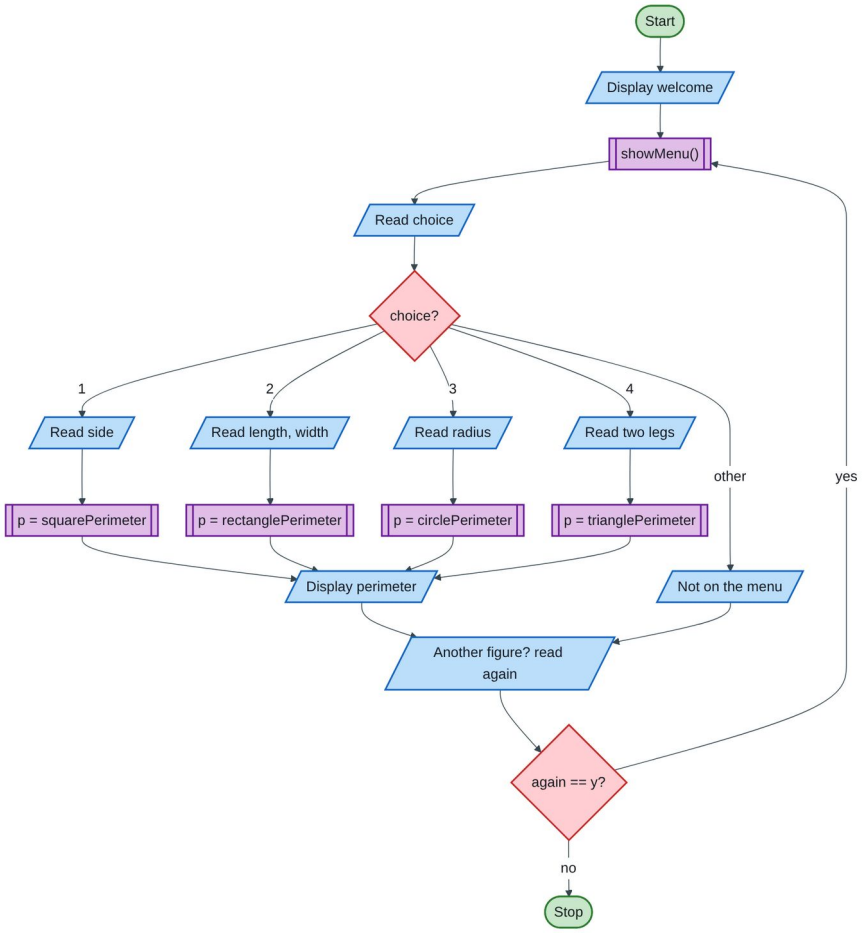


Figure 5.4 — Flowchart

Code (H-D-I-P-O-R):

```c
/* H: interactive perimeter calculator for four figures, built from functions */
#include <stdio.h>
#include <math.h>
#define PI 3.14159

void   showMenu(void);
double squarePerimeter(double a);
double rectanglePerimeter(double l, double w);
double circlePerimeter(double r);
double trianglePerimeter(double a, double b);

int main(void)
{
    int choice;                                 /* D */
    char again;
    double p, x, y;

    printf("Hello! I will work out perimeters for you.\n");
    do {
        showMenu();
        printf("Which figure would you like (1-4)? ");    /* I */
        scanf("%d", &choice);

        switch (choice) {                       /* P */
        case 1:
            printf("What is the length of one side? ");
            scanf("%lf", &x);
            p = squarePerimeter(x);
            break;
        case 2:
            printf("Please give the length and the width: ");
            scanf("%lf %lf", &x, &y);
            p = rectanglePerimeter(x, y);
            break;
        case 3:
            printf("What is the radius? ");
            scanf("%lf", &x);
            p = circlePerimeter(x);
            break;
        case 4:
            printf("Give the two legs (adjacent and opposite): ");
            scanf("%lf %lf", &x, &y);
            p = trianglePerimeter(x, y);
            break;
        default:
            printf("Sorry, that is not on the menu.\n");
            p = -1;
        }
        if (p >= 0)
            printf("The perimeter is %.2f units.\n", p);  /* O */

        printf("Would you like another figure (y/n)? ");
        scanf(" %c", &again);                   /* space skips the Enter key */
    } while (again == 'y' || again == 'Y');

    printf("Goodbye!\n");
    return 0;                                   /* R */
}

void showMenu(void)
{
```

```
    printf("\n------ PERIMETER MENU ------\n");
    printf("1. Square\n2. Rectangle\n3. Circle\n4. Right-angled triangle\n");
}

double squarePerimeter(double a)              { return 4 * a; }
double rectanglePerimeter(double l, double w) { return 2 * (l + w); }
double circlePerimeter(double r)              { return 2 * PI * r; }
double trianglePerimeter(double a, double b)  { return a + b + sqrt(a * a + b * b); }
```

Tests: square, side 5 → 20.00; rectangle 4 × 3 → 14.00; circle, r = 2 → 12.57; triangle with legs 3 and 4 → 3 + 4 + 5 = 12.00.

Marking guide [20]: conversational prompts (requirement i) [2]; menu shown and choice read (ii) [2]; the four perimeter functions with correct formulas (iv) [6]; `switch` asking only for the inputs each figure needs [3]; repeat-or-stop loop (iii) [3]; invalid choice handled [1]; flowchart [3].

Engineering lens: in open-channel hydraulics the important perimeter is the **wetted perimeter**, only the part of the boundary touching water. For a rectangular channel of bed width b and flow depth y it is P = b + 2y (the top is open). Adding a fifth menu option for it, together with the flow area A = by and hydraulic radius R = A/P, turns this exam program into a real design tool.

Examiner's tip: read the requirements list (i)–(iv) and tick each off in comments; examiners mark against exactly that list.

Question 6 [20]

6(a) Semantic error, with an example [2]

Like you're 3: "The cup drank the milk" has perfect grammar but makes no sense. That is a semantic error.

A semantic error is a statement that obeys C's grammar but has an invalid or unintended **meaning**, e.g. wrong types or misuse of an operator [1]. Example: `int count = "five";` (text put in an integer), or `printf("%d", 2.5);` (integer format for a double) [1].

6(b) Syntax of the switch statement [3]

```
switch (expression) {          /* int or char expression */
    case constant1:
        statements;
        break;                  /* leave the switch */
    case constant2:
        statements;
        break;
    default:                    /* none of the cases matched */
        statements;
}
```

Marks: `switch (expression)` with braces [1]; `case` labels with `break` [1]; `default` [1].

6(c) Valid or invalid? [5]

Statement	Verdict	Reason [1 each]
(i) `V = 2R;`	**Invalid**	`2R` is not a valid identifier or expression; multiplication needs `*`: `V = 2 * R;`
(ii) `print("I am in Bulawayo");`	**Invalid**	There is no `print` function in C; it must be `printf`
(iii) `scanf("%f", y);`	**Invalid**	Missing `&`: `scanf("%f", &y);`
(iv) `if( choice=2) printf(" Play Chess");`	**Invalid (logic)**	Compiles, but `=` assigns instead of comparing, so it is always true; should be `==`
(v) `++k;`	**Valid**	Pre-increment: adds 1 to k

6(d) Program: each value from the largest down, with count and positions [10]

Like you're 3: in a row of cards, find the biggest number, say how many cards have it and where they are. Then find the biggest of the numbers that are left, and so on down to the smallest.

Algorithm:

```
Step 1: Start; read up to 10 positive integers into a[ ], count them as n
Step 2: prev = a very large number
Step 3: max = largest a[i] that is smaller than prev
Step 4: If no such value exists, stop
```

```
Step 5: Count how many a[i] equal max; display max, the count and each position i+1
Step 6: prev = max; go to Step 3
```

Flowchart:

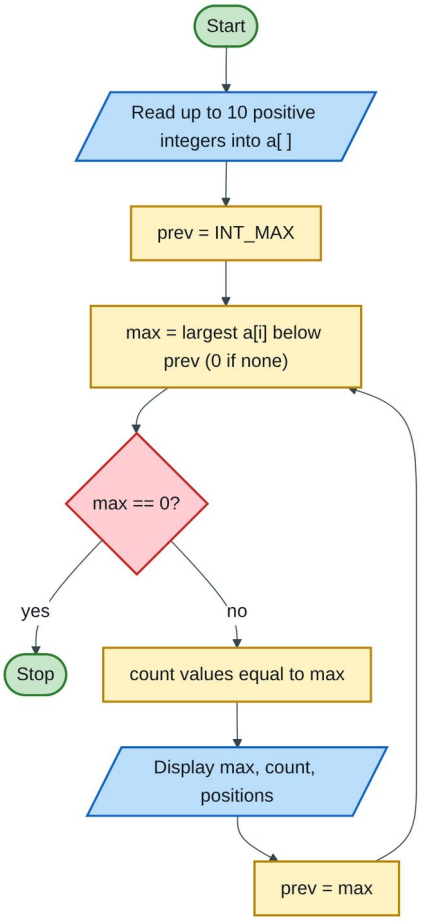


Figure 5.5 — Flowchart

Code (H-D-I-P-O-R):

```c
/* H: list each value from largest to smallest with its count and positions */
#include <stdio.h>
#include <limits.h>
#define MAX 10

int main(void)
{
    int a[MAX], n = 0, i, x, max, count, first, prev = INT_MAX;   /* D */

    printf("Enter up to 10 positive integers (0 to stop early):\n");
    while (n < MAX) {                                   /* I */
        scanf("%d", &x);
        if (x <= 0) break;
        a[n++] = x;
    }

    while (1) {                                         /* P */
        max = 0;
        for (i = 0; i < n; i++)                         /* largest value below prev */
            if (a[i] < prev && a[i] > max)
                max = a[i];
```

```
        if (max == 0) break;                            /* every value done */

        count = 0;
        for (i = 0; i < n; i++)
            if (a[i] == max) count++;

        printf("%d occurs %d times, at positions ", max, count);   /* O */
        first = 1;
        for (i = 0; i < n; i++)
            if (a[i] == max) {
                printf(first ? "%d" : ", %d", i + 1);
                first = 0;
            }
        printf("\n");
        prev = max;
    }
    return 0;                                           /* R */
}
```

Test with the paper's sample `7 10 143 10 52 143 72 10 143 7`:

```
143 occurs 3 times, at positions 3, 6, 9
72 occurs 1 times, at positions 7
52 occurs 1 times, at positions 5
10 occurs 3 times, at positions 2, 4, 8
7 occurs 2 times, at positions 1, 10
```

Marking guide [10]: reading up to 10 values into an array [2]; finding the maximum [2]; counting occurrences [2]; printing positions (1-based) [2]; repeating for the next largest value until done [2].

Engineering lens: the same "rank and locate" logic finds the **highest flood levels** in a record and the dates on which they occurred, the starting point for flood-frequency analysis.

Question 7 [20]

7(a) What is an infinite loop? Example [2]

Like you're 3: a song with no last verse: it goes round and round forever.

An infinite loop is a loop whose condition never becomes false, so it never ends [1]. Example: `int i = 1; while (i <= 5) printf("%d", i);` (i is never increased), or a deliberate `for (;;)` [1].

7(b) Output of the nested loop [3]

```
for (a = 1; a <= 4; a++)
{ for (b = 1; b <= 4 - a; b++)
      printf("%c", 'w' + b);
  printf("\n "); }
```

Like you're 3: 'w' + 1 is the letter after w, which is x; + 2 is y; + 3 is z.

a	b runs	Characters printed
1	1, 2, 3	xyz
2	1, 2	xy
3	1	x
4	none (4 − 4 = 0)	nothing

Each pass then prints `"\n "`: a new line **followed by a space**, so lines 2 onward start with a space:

```
xyz
 xy
 x

```

The last line holds only a space. Marks: xyz [1], xy and x on new lines [1], noticing the leading space and the empty fourth line [1].

7(c) Program: rectangle area using pointers [5]

Like you're 3: instead of handing over the toys, you hand over a note saying where each toy is kept, and the program goes and finds them.

```
/* H: area of a rectangle, reading and calculating through pointers */
#include <stdio.h>

int main(void)
{
    double length, width, area;              /* the actual boxes */
    double *pL = &length;                    /* pointer = address of length */
    double *pW = &width;
    double *pA = &area;

    printf("Enter the length: ");
    scanf("%lf", pL);                        /* pL is already an address: no & */
```

```c
    printf("Enter the width: ");
    scanf("%lf", pW);

    *pA = (*pL) * (*pW);                    /* area = length x width, via pointers */

    printf("Area of the rectangle = %.2f\n", *pA);
    return 0;
}
```

Marks: pointers declared and set with & [2]; input through pointers [1]; calculation with * dereference [1]; comments and output [1].

7(d) Program: help a child count with asterisks [10]

Like you're 3: you say a number, the computer draws that many stars; if you say a silly number it says "oops", and it keeps playing until you want to stop.

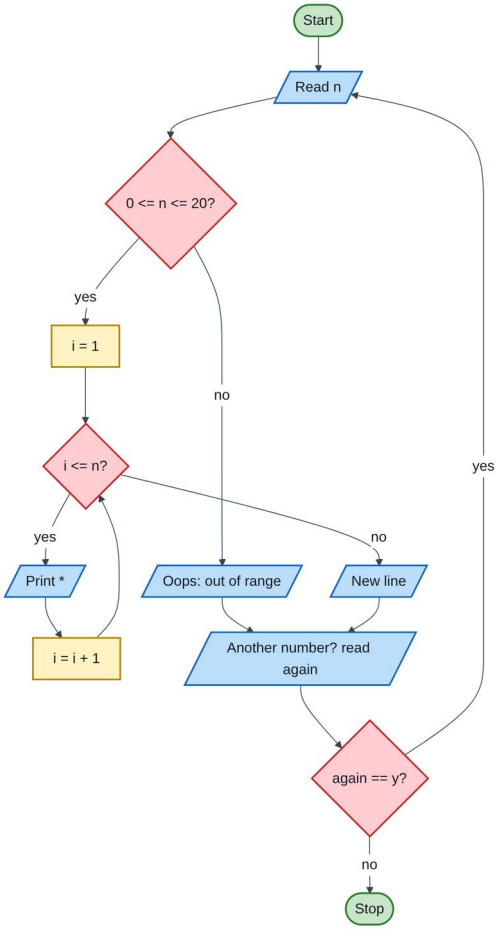


Figure 5.6 — Flowchart

```c
/* H: counting helper - prints n asterisks for 0 <= n <= 20, until the child stops */
#include <stdio.h>

int main(void)
{
    int n, i;                                   /* D */
    char again;

    printf("Let's count with stars!\n");
    do {
```

```c
        printf("Type a number from 0 to 20: ");   /* I */
        scanf("%d", &n);

        if (n < 0 || n > 20) {                     /* P: validation */
            printf("Oops! %d is not between 0 and 20.\n", n);
        } else {
            for (i = 1; i <= n; i++)               /* O: one star per count */
                printf("*");
            printf("   <- %d star(s)\n", n);
        }

        printf("Another number? (y/n): ");
        scanf(" %c", &again);
    } while (again == 'y' || again == 'Y');

    printf("Well done! Goodbye!\n");
    return 0;                                      /* R */
}
```

Test: 5 → *****; 0 → an empty line of stars; 25 → “Oops!”.

Marking guide [10]: repeat-until-done loop [3]; range check with an error message [2]; star-printing loop [3]; friendly prompts [1]; clean ending [1].

Sources for this book

- [MIT-1] MIT 6.087 — integer division, strings, pointers and switch in C (Q4, Q6, Q7).
- [NUST-1] SCS 1101 course outline.
- Textbook knowledge (tier 4): number conversion, generations, printers, viruses, language types, concrete statistics (target mean strength margin), open-channel wetted perimeter.

Source register

The full register for the solution-book set. Sources marked ✓ are cited in this book; the rest (MIT, UCLA and Wits material used across the set) are further reading on the same topics.

Key	Source	Institution	Used for	In this book
NUST-1	SCS 1101 course outline (uploaded)	NUST	Syllabus, assessment, C as the course language	✓
MIT-1	6.087 Practical Programming in C	MIT	C syntax, pointers, structs, files, coding style	✓
MIT-2	6.006 Lecture 19: Dynamic Programming I (Fibonacci)	MIT	Why naive recursive Fibonacci is slow; memoization	Further reading
MIT-3	Whirlwind Computer, 1944–59 (IEEE Milestone)	MIT	Magnetic-core memory, real-time computing	Further reading
MIT-4	Wang et al., *Undefined Behavior: What Happened to My Code?*	MIT CSAIL	Run-time and undefined-behaviour errors	Further reading
UCLA-1	First internet message, 29 Oct 1969	UCLA Samueli	History of networks and the internet	Further reading
UCLA-2	Birthplace of the Internet (IEEE Milestone)	UCLA	ARPANET, packet switching, first node	Further reading
UCLA-3	CS 31 Introduction to Computer Science I, syllabus	UCLA	Intro-programming scope; portability across compilers	Further reading
WITS-1	Sithole (Wits) & Ruhode, *Cloud Computing Adoption: Opportunities and Challenges for SMMEs in South Africa*, arXiv:2108.10079	Wits	Cloud benefits and barriers in Southern Africa	Further reading

Key	Source	Institution	Used for	In this book
WITS-2	*Leadership and the adoption of cloud computing for enterprises in Johannesburg* (2022)	Wits	Drivers and barriers of cloud adoption	Further reading
STD-1	NIST SP 800-145, *The NIST Definition of Cloud Computing*	NIST	SaaS, PaaS, IaaS; 5 characteristics; 4 deployment models	Further reading
RES-1	Bakasa & Pekane, *Decision Criteria for Cloud Adoption by Large Enterprises in South Africa*, arXiv:2108.09757	UCT	Security and data sovereignty as selection criteria	Further reading
RES-2	Fu et al., *Error Log Analysis in C Programming Language Courses* (ICCE 2015)	Kyushu University	The errors novice C students actually make	Further reading